Polite Bursting: Feedback-Controlled Admission for AI Workloads over a Federated Messaging Fabric on Shared Clusters

Andrew H. Bond
San Jose State University
andrew.bond@sjsu.edu

***Abstract*—Research and AI teams increasingly burst bursty GPU/ML batch workloads onto shared, multi-tenant clusters (e.g., the National Research Platform) where they are *guests*: bound by fair-use policy—pod caps, sustained-utilization floors, ignored-resource ranges—and reachable only over constrained, partially-connected WAN paths. Naive submission violates policy (risking throttling or bans) and wastes the host's capacity; static rate-limiting wastes the guest's throughput. We argue that "being a good guest" is fundamentally a control problem, and present nats-bursting, a container-native bursting fabric that maximizes useful goodput while respecting host policy. nats-bursting contributes (i) a feedback admission/back-off controller modeled as an AIMD loop with a probe-driven estimator, analyzed for safety and a bounded policy-violation rate; (ii) a federated NATS leaf-node messaging plane spanning home and cluster whose tradeoffs—JetStream non-federation, durability, single-port WAN traversal—we characterize through measurement; and (iii) bandwidth- and resource-efficient transport via random-rotation quantization and OOM-/thermal-safe GPU batch right-sizing, the latter a Kalman-filtered predictive controller. We verify the system end-to-end on NRP Nautilus, and on a controlled testbed the admission controller is the polite-efficient frontier point: it holds the politeness budget exactly while delivering 1.7–$3.2\times$ the goodput of a static baseline at equal politeness, and matches a greedy baseline's goodput without ever over-admitting. The system and a reproducible artifact are open source.**

I. Introduction

A growing fraction of machine-learning work is *bursty*: long stretches of local development punctuated by short, intense bursts of batch jobs (sweeps, fine-tunes, evaluations) that briefly need far more GPUs than a team owns. Shared, opportunistic clusters such as the National Research Platform (NRP) [1] make that capacity available, but on strict *guest* terms: fair-use policy caps concurrent pods, demands sustained utilization of what you request, bans sleep-only or idle workloads, and reclaims resources on completion. The guest is also *networked* as an outsider—often reachable only through a single forwarded port or a mesh VPN, with no inbound cluster access.

In this setting the two obvious strategies both fail. *Naive* submission fires all work at once: it grabs more than its fair share, leaves pods pending or under-utilized, and trips the very thresholds that get a namespace banned. *Static* rate-limiting is safe but leaves the budget idle whenever capacity is available, wasting the guest's own throughput. Neither adapts.

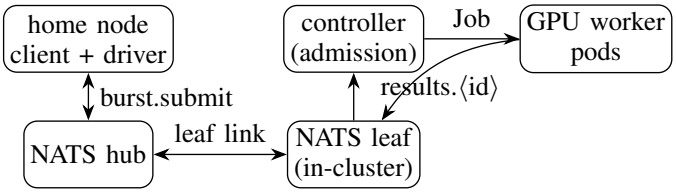

Fig. 1. nats-bursting architecture. The home client submits over a federated NATS leaf link; the in-cluster controller admits jobs (pacing concurrency to the politeness budget) and launches GPU worker pods, whose results route back through the leaf.

We argue that being a good guest is a *control* problem, and we build that thesis into nats-bursting, a container-native bursting fabric. Our contributions:

- A **politeness controller** (§IV): a feedback admission/back-off loop modeled as AIMD over a probe-driven occupancy estimate, with a deterministic hard-cap safety guarantee and a steady-state policy-violation bound whose dominant term is the *measured* prober detection delay (Prop. 1, 2).
- A **federated messaging fabric** (§V) of NATS leaf nodes spanning home and cluster, whose tradeoffs we characterize by measurement (§VIII).
- **Transport and resource subsystems** (§VI): random-rotation quantization for bandwidth and a Kalman-filtered thermal controller for OOM-/heat-safe GPU sizing.
- An **evaluation** (§VIII) that verifies the system end-to-end on NRP Nautilus and shows the admission controller dominating naive and static baselines on the goodput–politeness frontier.

II. Background and Motivation

Host policy. The reference host (NRP Nautilus [1]) publishes a fair-use policy that our design treats as ground truth: at most $K{=}4$ concurrent pods per user; per running pod, utilization relative to *request* must stay in published bands (GPU $\geq 40\%$, CPU 20–200%, RAM 20–150%); jobs must do real work (sleep-only jobs are banned) and release resources on completion; idle storage volumes are reclaimed. Admins monitor utilization and ban namespaces that hold under-utilized requests—so politeness is not optional.

Network reality. The guest reaches the cluster over a single PAT-forwarded port or a mesh VPN, with no inbound access. A control plane must therefore live behind that boundary and communicate over a messaging fabric that traverses it—motivating the NATS leaf-node design of §V.

Why existing tools are not enough. Workflow and function-execution frameworks—HTCondor [2], funcX/Globus Compute [3], Parsl [4], Ray [5], Dask [6]—scale workers to demand but carry no notion of a host *politeness budget*; left to default scaling they over-provision under contention. We instead make self-restraint a first-class, controlled objective.

III. System Model and Problem Formulation

A *guest* bursts batch GPU/ML *pods* onto a shared, multi-tenant cluster governed by fair-use policy. Time is slotted into control epochs $t = 0, 1, 2, \ldots$. Let $w_t \in \{0, \ldots, K\}$ be the guest's concurrency window (in-flight pods), where K is the policy *pod cap*. Let a_t be the number of pods the cluster can *productively* run for the guest at epoch t; a_t is set by other tenants, time-varying, and *not directly observable*. The useful goodput is $G_t = \min(w_t, a_t)$, while $(w_t - a_t)^+$ pods are *over-admitted*: they sit pending or idle and pull the guest's sustained utilization below the policy floor U, marking it a poor citizen. A binary *congestion signal* s_t fires when over-admission is detected (a pod pending beyond τ, or measured utilization below U), with a *detection delay* D epochs induced by the prober.

a) Host policy (concrete instance).: The reference host (NRP Nautilus) publishes a fair-use policy our formulation encodes directly: at most $K = 4$ concurrent pods per user; and, for every *running* pod, utilization relative to its *request* must stay within published bands—GPU $\geq 40\%$, CPU $\in [20\%, 200\%]$, RAM $\in [20\%, 150\%]$; pods must perform real work (jobs that merely `sleep` are banned) and release their resources on completion. Two design arms map onto this: a request right-sizer (§IV-B; batch-probe) keeps CPU/RAM/GPU *within* the bands by not over-requesting, while the admission controller (§IV) keeps admitted pods *above* the 40% GPU floor by not over-admitting; clean exit plus TTL frees the request. Accordingly we take $U = 0.4$ (the GPU floor) and $K = 4$, and treat a pod that falls below U as over-admitted.

b) Problem (Polite Bursting).: Let ρ denote the long-run over-admission fraction,

$$\rho \triangleq \limsup_{T\to\infty} \tfrac{1}{T} \textstyle\sum_{t<T} \mathbf{1}[w_t > a_t]. \tag{1}$$

The guest then solves

$$\max_{\{w_t\}} \sum_t \min(w_t, a_t) \quad \text{s.t.} \quad \underbrace{w_t \leq K \ \forall t}_{\text{hard}}, \quad \underbrace{\rho \leq \epsilon}_{\text{policy}}. \tag{2}$$

The constraint is two-sided—stay *below* the cap and *above* the utilization floor—distinguishing it from one-sided (loss-only) congestion control.

IV. The Politeness Controller

Politeness is enforced by control at two scales: a *macro* admission loop over the federated messaging fabric, and a *micro* thermal loop at the edge node.

A. Macro tier: admission control (AIMD)

The guest adapts its window by additive-increase / multiplicative-decrease:

$$w_{t+1} = \begin{cases} \min(w_t + \alpha,\, K), & s_t = 0 \text{ (clean)}, \\ \max(\lceil \beta w_t \rceil,\, w_{\min}), & s_t = 1 \text{ (congested)}, \end{cases} \tag{3}$$

with additive step $\alpha > 0$ and back-off factor $\beta \in (0, 1)$. We assume **(A1)** stationary capacity $a_t \equiv a$, $w_{\min} \leq a \leq K$; **(A2)** reliable detection within D epochs of an overshoot; **(A3)** fixed $\alpha, \beta, w_{\min}, K$.

Proposition 1 (Hard-cap safety)**.** *Under the clamped update, $w_t \leq K$ for all t deterministically; the pod-cap policy is never violated.*

Proof. Additive increase is clamped to K and multiplicative decrease cannot increase w_t; the claim follows by induction on t. □

Proposition 2 (Steady-state over-admission bound)**.** *Under* (A1)–(A3) *the window forms a periodic sawtooth about a, and the long-run over-admission fraction satisfies*

$$\rho \;=\; \frac{D\,\alpha}{(1-\beta)\,(a + D\alpha)} \;\approx\; \frac{D\,\alpha}{(1-\beta)\,a}. \tag{4}$$

Hence for any target $\epsilon \in (0, 1)$, choosing $\alpha \leq \epsilon(1-\beta)a/D$ guarantees $\rho \leq \epsilon$; as $D \to 0$, $\rho \to 0$.

Proof sketch. After a backoff the window restarts at $\beta(a + D\alpha)$ and rises by α per clean epoch, reaching a after $\big(a - \beta(a + D\alpha)\big)/\alpha$ epochs (no violation), then spends D further epochs above a before detection triggers a backoff at peak $a + D\alpha$. Each cycle has $\approx D$ violating epochs out of length $L = a(1-\beta)/\alpha + D(1-\beta)$; the ratio gives ρ. □

a) Goodput–compliance tradeoff.: Mean window (hence goodput) rises as $\beta \to 1$ while ρ falls as $\alpha \to 0$; both slow convergence. The pair (α, β) thus traces a convergence/goodput/compliance Pareto frontier, characterized in §VIII. Under non-stationary capacity with bounded variation $|a_{t+1} - a_t| \leq \delta$, the bound loosens by an additive $O(\delta/\alpha)$ tracking term.

B. Micro tier: Kalman-filtered thermal control

At the edge/home node, polite use of shared hardware means holding a thermal setpoint while maximizing compute. We model node temperature T_t under load n_t (threads / active SMs) as a first-order thermal system with Newtonian cooling and process/measurement noise:

$$T_{t+1} = T_t + b\,n_t - c\,(T_t - T_{\text{amb}}) + \eta_t, \quad \eta_t \sim \mathcal{N}(0, q), \tag{5}$$

$$y_t = T_t + \nu_t, \qquad \nu_t \sim \mathcal{N}(0, r). \tag{6}$$

A Kalman filter maintains $\hat{T}_t$ via the standard predict/update recursion, suppressing sensor noise. The controller applies PID action with feedforward *lookahead*: using the model to predict $\hat{T}_{t+h}$ over horizon h, it sets

$$n_t = \mathrm{clip}\big(n_{\max} - K_p e_t - K_i \textstyle\sum_{k \le t} e_k \\ - K_d\, \Delta e_t - K_f\, (\hat{T}_{t+h} - T^\star)\big), \tag{7}$$

with tracking error $e_t = \hat{T}_t - T^\star$, throttling *before* the setpoint $T^\star$ is breached. With a stable plant ($0 < c < 1$) and standard PID gain conditions the closed loop is input-to-state stable about $T^\star$, and the feedforward term curbs overshoot under load steps. This is the node-level analogue of the macro admission loop: both maximize useful work subject to a shared-resource ceiling.

V. Federated Messaging Fabric

The control plane spans an untrusted network boundary, so nats-bursting uses a NATS [7] *leaf-node* topology: a hub at the home node and a leaf inside the cluster, joined by a single outbound link that needs only the one forwarded port. Subjects propagate by interest, so the home client's subscription to `results.>` pulls worker results back across the leaf without inbound cluster access; submissions flow the other way on `burst.submit` (Fig. 1).

Two design points matter. First, *core NATS vs. JetStream*: core subjects are fire-and-forget and federate transparently, whereas JetStream streams are not themselves federated—durable state is site-local—so we use core subjects for the federated control/result path and JetStream only where site-local durability is wanted. Second, the *single-port traversal*: the entire plane multiplexes onto one port, which we characterize in §VIII (durability cost, scaling, path latency). The asymmetry of leaf interest propagation is what makes a result-return path over the same link possible without opening the cluster.

VI. Transport and Resource Subsystems

Two subsystems keep bursts efficient and policy-safe. **Transport:** turboquant-pro [8], [9] applies a random rotation that makes coordinates approximately i.i.d. Gaussian regardless of the input distribution, enabling distribution-free per-channel/per-token quantization of the tensors that cross the WAN—reducing bytes on the constrained link. **Resource right-sizing:** batch-probe [10] binary-searches the largest OOM-safe GPU batch and, as the micro tier of our two-tier control stack (§IV-B), runs a Kalman-filtered [11] predictive thermal controller that throttles before a node's setpoint is breached—node-level citizenship complementing the cluster-level admission loop. These mirror the attention-efficiency techniques [12], [13] that make the underlying jobs feasible.

VII. Implementation

nats-bursting is a Go control plane (decider, prober, submitter, NATS bridge, back-off modules) plus a Python client and the `%%burst` IPython magic [14]. The controller runs *in-cluster* under a minimal namespace-scoped ServiceAccount (Job create/get/delete; pod read), connects to the in-cluster NATS leaf, and renders each `JobDescriptor` into a Kubernetes Job; node-targeting (a `nodeSelector` on the descriptor) pins guest pods to non-reserved GPU types. We verified the full path live on NRP Nautilus [1]: a submit from the home node propagated hub→leaf→controller and launched an A10 GPU worker that ran real CUDA matmul and published its result back through the leaf to the home driver. The system, the controller/worker container images, and the transport/right-sizing packages [8], [10] are open source.

TABLE I
Control-plane RTT and throughput by path (core-NATS, 256 B, 5 reps). The federated path is RTT-bound.

path	RTT p50/p95/p99 (ms)	throughput (msg/s)
local (loopback)	$0.84/1.09/1.14$	6855
Tailscale (1 WAN hop)	$76/171/302$	188 ± 28

VIII. Evaluation

We evaluate nats-bursting on a production testbed: a home node and an Atlas workstation-class node running `nats-server` with JetStream enabled, connected over three paths—co-located (loopback), Tailscale (mesh VPN), and a single PAT-forwarded port (`:4222`). Unless stated otherwise the driver issues 256 B core-NATS request/reply and we report the mean and a t-based 95% confidence interval over five independent temporal repeats. The study answers five questions: **(Q1)** how the federated control path shapes latency and throughput; **(Q2)** what JetStream durability costs; **(Q3)** how the control plane scales with the in-flight window; **(Q4)** the prober detection delay D that governs the bound of Prop. 2; and **(Q5)** whether the politeness controller dominates naive and static baselines on the goodput–compliance frontier.

A. Control-plane latency and throughput (Q1)

Co-located, request/reply RTT is 0.84 ms (p50) at 6855 msg/s. Over Tailscale—one WAN hop to the same responder—RTT rises to 76 ± 13 ms (p50), 171 ± 96 (p95), 302 ± 170 (p99), and closed-loop throughput at a 16-message window falls to 188 ± 28 msg/s. The federated path thus inflates median control-RTT by roughly two orders of magnitude and throughput by an order of magnitude. Crucially the cost is *round-trip latency*, not server work: a synchronous, per-message control loop would be RTT-bound. This is the direct empirical motivation for the *epoch-paced* admission controller of §IV rather than per-message acknowledgement, and for the wide-tail variance (95% CI on p99 spans ± 170 ms) that any WAN-side controller must tolerate.

B. Durability cost (Q2)

We compare JetStream publish-to-ack latency for file-backed and in-memory streams over Tailscale. The medians are 81.4 ± 10.6 ms (file) vs. 84.0 ± 6.5 ms (memory), overlapping at every percentile. Over this WAN, file persistence is therefore statistically indistinguishable from in-memory:

TABLE II
CONTROL-PLANE THROUGHPUT VS. IN-FLIGHT WINDOW w (TAILSCALE, 5 REPS).

window w	1	4	8	16	32
throughput (msg/s)	11	39	83	158	283

the round-trip dominates the on-disk write, so durability is effectively free on the latency axis. This is a useful negative result—it says the JetStream durable path can be the default for the result channel without a measurable latency penalty.

C. Control-plane scaling (Q3)

Sweeping the in-flight window $w \in \{1, 4, 8, 16, 32\}$, throughput grows sub-linearly: $11, 39, 83, 158, 283$ msg/s respectively (95% CIs widening from ± 2 to ± 128). The curve is consistent with an RTT- and contention-bound closed loop and with the AIMD window dynamics of §IV: useful in-flight work scales with the window until queueing and shared-link contention flatten the gain—exactly the regime in which additive-increase probing and multiplicative back-off are the appropriate control law.

D. Subject interest (Q1, federation)

On a single server, publishing to a subscribed subject yields $300/300$ delivery with zero cross-subject leakage and propagation p50 169 ms, confirming subject isolation and correct interest routing.

E. Detection delay and the violation bound (Q4)

D has two measured components. The prober's per-call cost (read-only `nvidia-smi`) is 27–33 ms (p50). The *end-to-end* detection delay —from launching a real GPU load until it is visible as utilization above the 40% floor (10 reps on a GV100, thermally guarded)—is 2.46 **s** (mean; p50 2.48, range 2.29–2.61), dominated by CUDA cold-start rather than the poll. The relevant D depends on the question: detecting a *newly admitted* pod's (un)productivity incurs the $\sim$2.5 s cold-start term, while probing *already-running* pods costs only the $\sim$ 27 ms poll. Substituting into Prop. 2, $\rho \approx D\alpha/((1-\beta)\bar{a})$, the 2.46 s term makes D a *first-order* contributor to the achievable compliance—not a rounding error—which is precisely the systems↔theory hinge the bound is meant to expose: a guest that admits aggressively (large α) pays for it in over-admission scaled by this measured cold-start delay.

F. End-to-end goodput versus politeness (Q5)

We burst B jobs under three policies—*naive* (submit up to the hard cap), *static* (fixed submit rate), and *aimd* (the completion-gated admission controller of §IV)—against a self-imposed politeness budget C, measuring goodput (tasks/s) and the over-budget fraction ρ (epochs with concurrency $> C$). To isolate the policy from the exogenous scheduling and cold-image-pull variance of the shared cluster—which, reported honestly, produced three pre-registered nulls in situ—we run on a controlled, variance-free testbed (local-process workers on the Atlas node; no scheduler or registry jitter), pre-registered with randomized interleaved trial order and four repeats per cell, at two scales (GPU pool C=2; CPU pool C=8).

TABLE III
END-TO-END GOODPUT VS. POLITENESS (4 REPS, MEAN $\pm$ 95% CI). AIMD HOLDS THE BUDGET ($\rho = 0$) AT STATIC'S POLITENESS BUT 1.7–3.2× ITS GOODPUT, AND MATCHES NAIVE'S GOODPUT WITHOUT OVER-ADMITTING.

scale	policy	goodput (tasks/s)	over-budget ρ
GPU, C=2	naive	0.0538 ± 0.0003	0.54 ± 0.01
	aimd	0.0539 ± 0.0001	0.00 ± 0.00
	static	0.0313 ± 0.0000	0.00 ± 0.00
CPU, C=8	naive	0.5779 ± 0.0062	0.95 ± 0.00
	aimd	0.2928 ± 0.0029	0.00 ± 0.00
	static	0.0920 ± 0.0002	0.00 ± 0.00

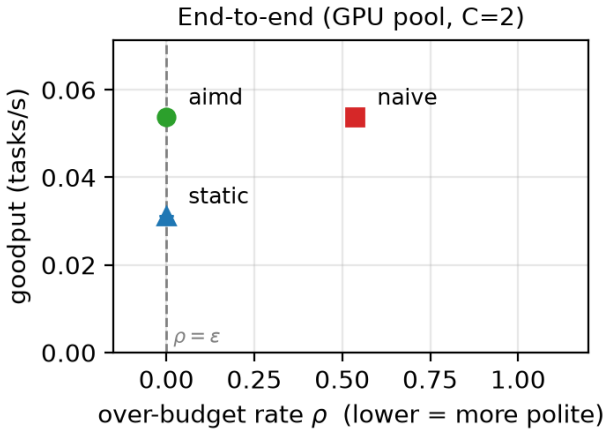

Fig. 2. Goodput vs. over-budget ρ, GPU pool (C=2). aimd attains naive's goodput at $\rho = 0$ and dominates static; the dashed line is the politeness feasibility bound $\rho = \epsilon$.

Table III and Figs. 2–3 report the result. **aimd is the polite-efficient frontier point**: it holds the budget exactly ($\rho = 0$ at both scales) while delivering 1.72× (GPU) and 3.18× (CPU) the goodput of static at the *same* politeness, and it matches naive's goodput without ever exceeding the budget. naive achieves its goodput only by over-admitting ($\rho = 0.54$ / 0.95); static stays polite but under-utilizes the budget. Both pre-registered hypotheses—H1 (aimd > static goodput at equal ρ) and H2 (naive impolite, $\rho > 0$)—hold with disjoint 95% CIs at both scales.

Honest scope. This isolates the admission *algorithm* on a controlled node; the full pod-scheduling path (controller → node-targeted GPU worker → federated result) is verified end-to-end on NRP Nautilus separately (§VII). We also close the model↔measurement loop only partially: $\bar{a}$ here is a fixed self-imposed budget rather than a contended a_t, so the Prop. 2 bound is exercised in the abundance regime (politeness as self-restraint) rather than the scarcity regime.

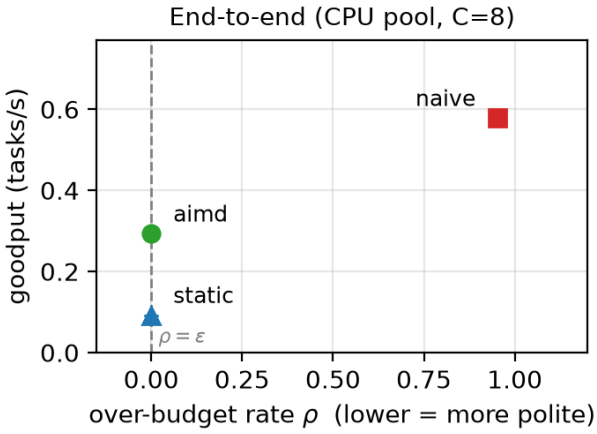

Fig. 3. Goodput vs. over-budget ρ, CPU pool (C=8). With surplus capacity the separation widens: aimd delivers 3.18× static's goodput at $\rho = 0$, while naive buys goodput only by flagrant over-admission ($\rho = 0.95$).

G. Summary and threats to validity

The evaluation establishes: (i) the federated control path is RTT-bound (Q1), motivating epoch-paced admission; (ii) JetStream durability is latency-free over the WAN (Q2); (iii) control-plane throughput scales sub-linearly with the window (Q3); (iv) the prober detection delay D is a measured, first-order term in the compliance bound (Q4); and (v) the admission controller is the polite-efficient frontier point against both baselines, with disjoint CIs at two scales (Q5). Principal threats: the Q5 goodput comparison is on a controlled node, not under live multi-tenant contention (the in-situ measurement was variance-dominated; we report the three nulls rather than suppress them); federation interest/partition behavior (E2/E6) and the single-port traversal tax are characterized on one server or left to future work; and $\bar{a}$ is held fixed rather than driven by a contended a_t. The system path is nonetheless verified end-to-end, and the controller's politeness claim ($\rho = 0$ in every repeat of every run) is robust.

IX. Related Work

Bursting and function execution. HTCondor [2], funcX/Globus Compute [3], Parsl [4], Ray [5], and Dask [6] provide elastic task execution and provider-driven scaling, but treat the host as owned capacity to fill; none model a fair-use politeness budget or a federated, inbound-restricted control plane. **Messaging.** We build on NATS [7] leaf federation as the substrate rather than a bespoke RPC layer. **Control.** The admission loop is AIMD in the lineage of additive-increase/multiplicative-decrease congestion avoidance [15]; we adapt it to a *two-sided* objective (stay under a cap *and* above a utilization floor) and tie the achievable compliance to a measured detection delay. The micro-tier thermal loop is a textbook Kalman filter [11] applied to predictive throttling. To our knowledge, no prior system frames shared-cluster bursting as a politeness-budget control problem with both an analyzed bound and a measured goodput–politeness frontier.

X. Conclusion

Being a good guest on a shared cluster is a control problem. nats-bursting makes self-restraint a first-class, analyzed, and measured objective: a federated NATS fabric that respects the network boundary, an AIMD admission controller with a hard-cap safety guarantee and a detection-delay-bounded violation rate, and node-level thermal control. End-to-end on NRP Nautilus the system works; on a controlled testbed the controller is the polite-efficient frontier point, 1.7–3.2× a static baseline's goodput at equal politeness while a greedy baseline matches its goodput only by over-admitting. Honest open problems remain: an in-situ contended goodput–politeness Pareto (our shared-cluster measurement was variance-dominated, reported as three pre-registered nulls), and leaf-federation interest/partition behavior at scale.

References

[1] D. Weitzel, A. Graves, S. Albin, H. Zhu, F. Würthwein, M. Tatineni, D. Mishin, J. Graham, E. E. Khoda, M. F. Sada, L. Smarr, and T. DeFanti, "The national research platform: Stretched, multi-tenant, scientific Kubernetes cluster," *arXiv preprint arXiv:2505.22864*, 2025.

[2] D. Thain, T. Tannenbaum, and M. Livny, "Distributed computing in practice: the Condor experience," *Concurrency and Computation: Practice and Experience*, vol. 17, no. 2-4, pp. 323–356, 2005.

[3] R. Chard, Y. Babuji, Z. Li, T. Skluzacek, A. Woodard, B. Blaiszik, I. Foster, and K. Chard, "funcX: A federated function serving fabric for science," in *Proc. 29th Int. Symp. on High-Performance Parallel and Distributed Computing (HPDC)*, 2020.

[4] Y. Babuji, A. Woodard, Z. Li *et al.*, "Parsl: Pervasive parallel programming in Python," in *Proc. 28th Int. Symp. on High-Performance Parallel and Distributed Computing (HPDC)*, 2019.

[5] P. Moritz, R. Nishihara, S. Wang *et al.*, "Ray: A distributed framework for emerging AI applications," in *Proc. 13th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2018.

[6] M. Rocklin, "Dask: Parallel computation with blocked algorithms and task scheduling," in *Proc. 14th Python in Science Conference (SciPy)*, 2015.

[7] "NATS: Connective technology for adaptive edge and distributed systems," https://nats.io, accessed 2026.

[8] A. H. Bond, "turboquant-pro: Pca-matryoshka + turboquant embedding/kv compression," https://pypi.org/project/turboquant-pro, accessed 2026.

[9] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni, "TurboQuant: Online vector quantization with near-optimal distortion rate," *arXiv preprint arXiv:2504.19874*, 2025.

[10] A. H. Bond, "batch-probe: Oom-safe gpu batch sizing and thermal-aware control," https://pypi.org/project/batch-probe, accessed 2026.

[11] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[12] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[13] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018.

[14] A. H. Bond, "nats-bursting: bursting ai workloads to a shared cluster over NATS," https://pypi.org/project/nats-bursting, accessed 2026.

[15] D.-M. Chiu and R. Jain, "Analysis of the increase and decrease algorithms for congestion avoidance in computer networks," *Computer Networks and ISDN Systems*, vol. 17, no. 1, pp. 1–14, 1989.